Работа с библиотекой requests, http-запросы.

Что такое HTTP?

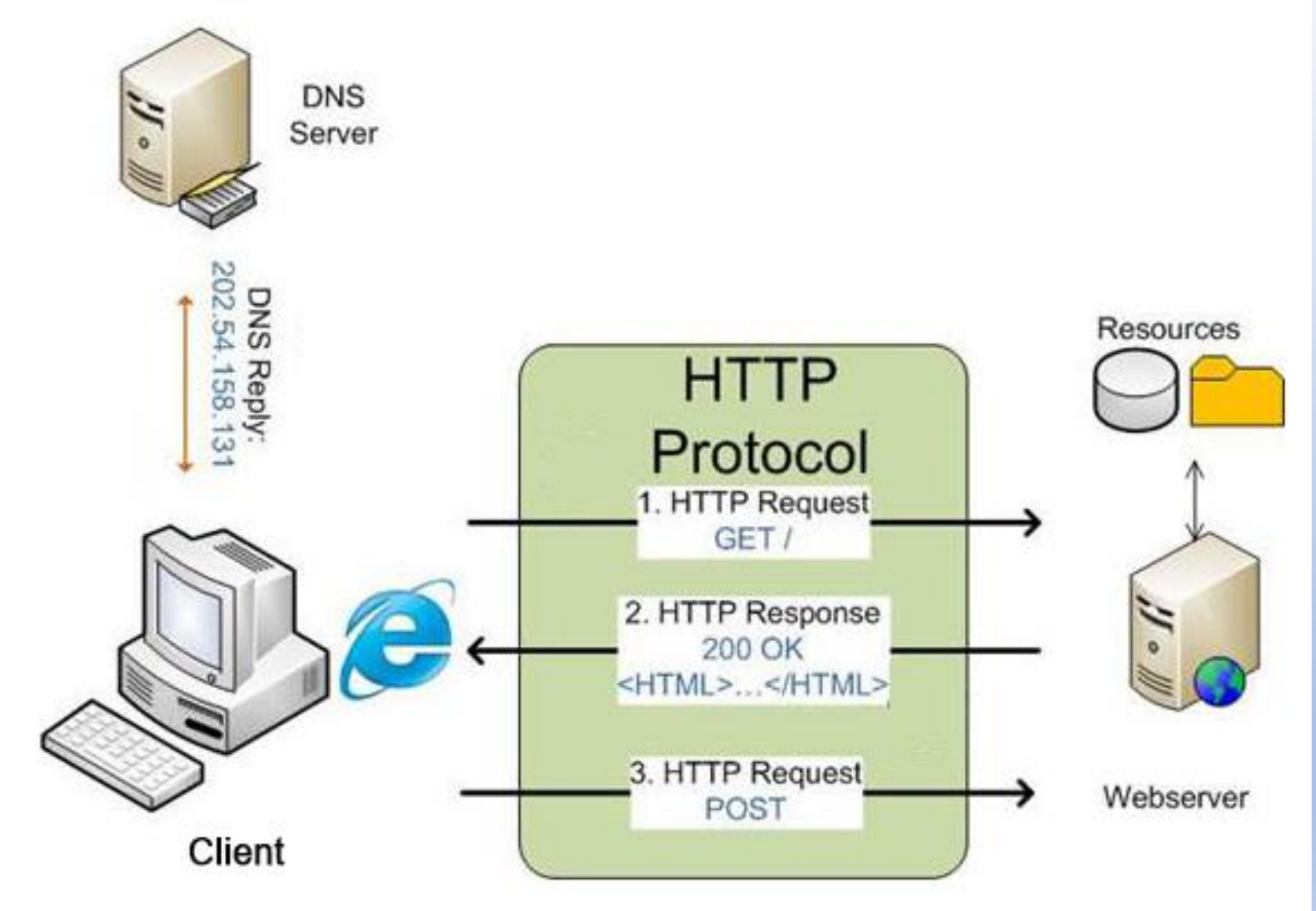
- HTTP (Hypertext Transfer Protocol) – это протокол для передачи данных (текстов, изображений, видео, аудио и т.д.) по интернету.
- HTTP не хранит состояние между запросами. Каждый запрос интерпретируется независимо от других запросов. Если нужно сохранять и передавать состояние – это должны делать реализации клиента и сервера

Стек протоколов TCP/IP

Прикладной уровень	**FTP** протокол передачи файлов	**HTTP** протокол передачи гипертекста	**SMTP** протокол передачи электронной почты		**Telnet** протокол, где администратор удаленно конфигурирует маршрутизатор	
Транспортный уровень	**TCP** протокол управления передачей				**UDP** протокол пользовательских дейтаграмм	
Сетевой уровень	**IP** протокол, который отвечает за продвижение пакета между сетями		**ICMP** протокол межсетевых управляющих сообщений		**RIP и OSPF** протоколы маршрутизации	**IGMP** протокол групповой адресации
Уровень сетевых интерфейсов	Ethernet, Fast E, Gigabit E	Token Ring	FDDI	ATM	X.25	Frame Relay

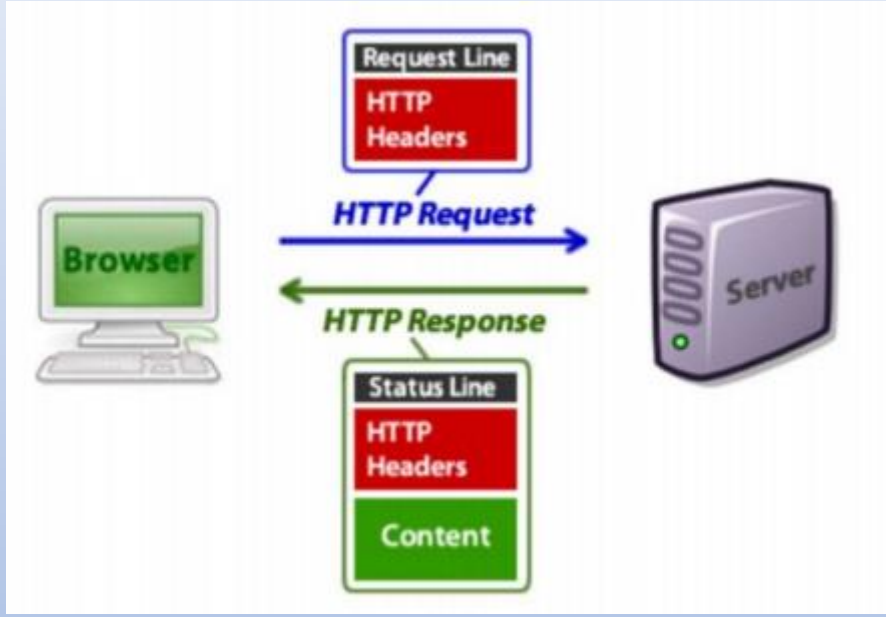
для локальных сетей

для глобальных сетей


DNS
Server
DNS Reply:
202.54.158.131
HTTP
Protocol
1. HTTP Request
GET /
2. HTTP Response
200 OK
<HTML>...</HTML>
3. HTTP Request
POST
Resources
Client
Webserver

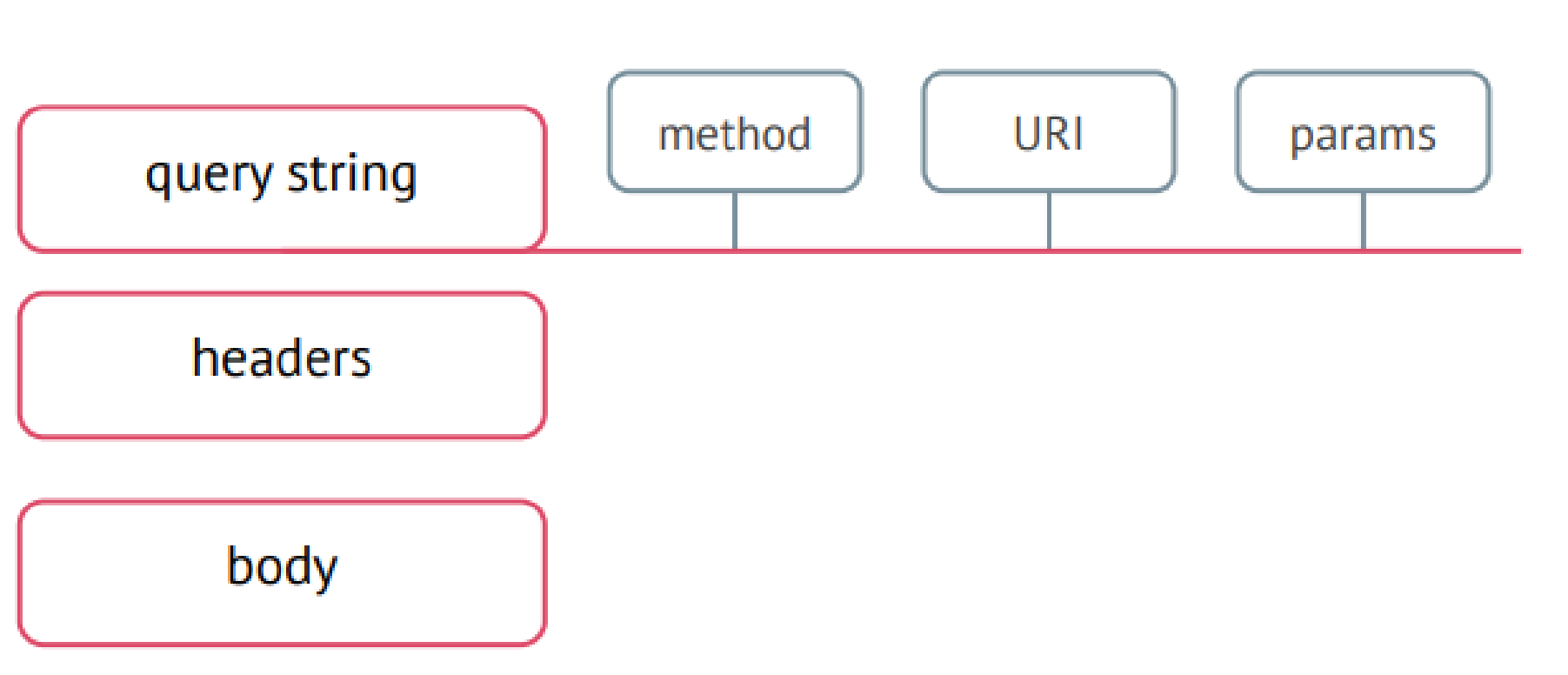
Client-Server

- Как выглядит клиент-серверное взаимодействие:



- В роли клиента может выступать не только браузер, но и любое устройство и даже другая программа. Когда говорят «клиент-сервер » , подразумевают, что есть сторона, которая запрашивает данные, а другая сторона ей отвечает.

HTTP Request (запрос)



Пример HTTP запроса

- GET /tutorials HTTP/1.1
- User-Agent: Mozilla/4.0 (compatible; MSIE5.01; Windows NT)
- Host: www.proselyte.net
- Accept-Language: ru-Ru
- Accept-Encoding: gzip, deflate
- Connection: Keep-Alive

Пример ответа сервера

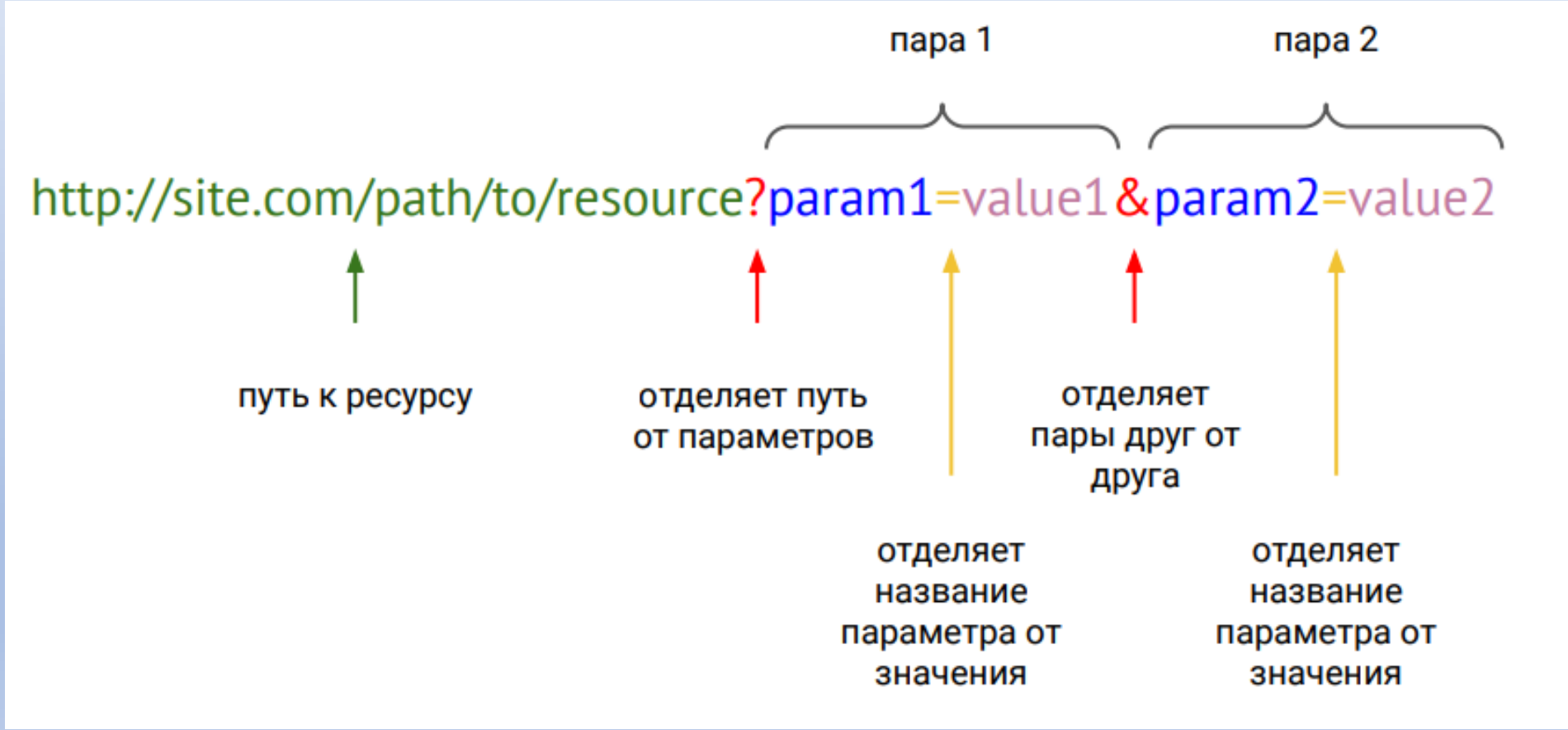
- HTTP/1.1 200 OK
- Date: Sun, 10 Feb 2013 03:51:41 GMT
- Content-Type: text/html; charset=UTF-8
- Transfer-Encoding: chunked
- Connection: keep-alive
- Keep-Alive: timeout=5
- Server: Apache
- X-Pingback: //webkyrs.info/xmlrpc.php
-
- <!DOCTYPE html PUBLIC «-//W3C//DTD XHTML 1.0 Transitional//EN» «http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd»>
- <html xmlns=»http://www.w3.org/1999/xhtml»>
- <head>
- <meta http-equiv=»Content-Type» content=»text/html; charset=utf-8” />
- <title>Документ без названия</title>
- </head>
- <body>
- </body>

HTTP методы

SAFE METHODS
NO ACTION ON SERVER
MESSAGE WITH
BODY
SEND DATA TO SERVER
GET
HTTP/1.1 MUST IMPLEMENT THIS METHOD
HEAD
INSPECT RESOURCE HEADERS
PUT
DEPOSIT DATA ON SERVER – INVERSE OF GET
POST
SEND INPUT DATA FOR PROCESSING
PATCH
PARTIALLY MODIFY A RESOURCE
TRACE
ECHO BACK RECEIVED MESSAGE
OPTIONS
SERVER CAPABILITIES
DELETE
DELETE A RESOURCE – NOT GUARANTEED

Параметры в адресе

- Структура URL



Заголовки

- Как правило, в заголовках передают информацию о запросе. Например, авторизационные данные пользователя, версию браузера, выставленные cookie, поддерживаемые форматы сжатия данных и т.д.
- Параметры запроса содержат информацию о том, что именно запрашивает пользователь.
- Также в параметрах запроса можно передавать произвольную информацию при размещении ссылки. Например, источник, где ссылка размещена (это используется в интернет-рекламе).

Пример HTTP ответа

```
HTTP/1.x 200 OK
Transfer-Encoding: chunked
Date: Sat, 28 Nov 2009 04:36:25 GMT
Server: LiteSpeed
Connection: close
X-Powered-By: W3 Total Cache/0.8
Pragma: public
Expires: Sat, 28 Nov 2009 05:36:25 GMT
Etag: "pub1259380237;gz"
Cache-Control: max-age=3600, public
Content-Type: text/html; charset=UTF-8
Last-Modified: Sat, 28 Nov 2009 03:50:37 GMT
X-Pingback: http://net.tutsplus.com/xmlrpc.php
Content-Encoding: gzip

Vary: Accept-Encoding, Cookie, User-Agent



<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Strict//EN" "http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd">

<html xmlns="http://www.w3.org/1999/xhtml">

<head>

<meta http-equiv="Content-Type" content="text/html; charset=utf-8" />

<title>Top 20+ MySQL Best Practices - Nettuts+</title>
```

Как делать запросы:

- POSTMAN

Postman

File Edit View Help

Home Workspaces API Network Reports Explore Search Postman Invite Upgrade

Overview GET https://http.cat/ No Environment

https://http.cat/ Save

GET https://http.cat/ Send

Params Authorization Headers (6) Body Pre-request Script Tests Settings Cookies

KEY	VALUE	DESCRIPTION
Postman-Token	<calculated when request is sent>	
Host	<calculated when request is sent>	
User-Agent	PostmanRuntime/7.28.4	
Accept	*/*	
Accept-Encoding	gzip, deflate, br	
Connection	keep-alive	
Key	Value	Description

Bulk Edit Presets

Body Cookies Headers (10) Test Results Status: 200 OK Time: 551 ms Size: 27.66 KB Save Response

Pretty Raw Preview Visualize HTML

```
<!DOCTYPE html>
<html lang="en">

<head>
    <meta charset="utf-8">
    <link href="/favicon.png" rel="shortcut icon">
    <link href="/apple-touch-icon.png" rel="apple-touch-icon-precomposed">
    <link href="mask-icon.svg" rel="mask-icon" color="#d0383e">
    <meta content="width=device-width,initial-scale=1" name="viewport">
    <meta content="Rogério Vicente" name="author">
    <meta content="An API for the awesome HTTP Cats! Use it in your website to show funny error messages."
```

Find and Replace Console Bootcamp Runner Trash

Библиотека requests

- Библиотека requests в настоящее время фактически является стандартом для реализации отправки HTTP-запросов в Python.
- Не является стандартной библиотекой.
- Установка:

pip install requests

Документация:

https://pythonru.com/biblioteki/kratkoe-rukovodstvo-po-biblioteke-python-requests

Выполнение простого GET-запроса:

```
import requests
url = "https://http.cat"
resp = requests.get(url)
print(resp)
print(resp.status_code)
print(resp.text)
```

```
<Response [200]>
200
<!DOCTYPE html><html lang="en"><head><meta charset="utf-8"><link href="/favicon.png" rel=
```

Атрибуты и методы объекта Response

- status_code – HTTP-статус ответа
- headers – заголовки ответа
- content – содержимое ответа в байтах
- text – содержимое ответа в текстовом представлении (utf8, так как все строки в Python – юникодные)
- json() – представление ответа в виде словаря. Работает только в том случае, если сервер вернул валидный JSON. Используется при работе с API

Параметры запроса и заголовки

- Чтобы передать параметры и заголовки в запросе, используйте именованные аргументы:
- Важно: конкретные значения параметров и заголовков зависят от сервера, к которому происходит обращение. Для того чтобы узнать точные значения параметров, нужно читать документацию или проверять опытным путем.

```
import requests
url = "https://site.org/get"
params = {"foo": "bar", "message": "hello"}
headers = {"Authorization": "secret-token-123"}
resp = requests.get(url, params=params, headers=headers)
```

Загрузка файлов

- Чтобы передавать файл, его нужно открыть в байтовом режиме и передать объект файла в параметр files:

```
import requests
with open('gifs/your_file.mp4', 'rb') as f:
    resp = requests.post('http://httpbin.org/post', files={"file": f})
```

Документация

- HTTP: https://habr.com/ru/post/215117/
- REQUESTS: https://digitology.tech/docs/requests/
- Полигон Яндекса: https://yandex.ru/dev/disk/poligon/

- STACKOWERFLOW: https://api.stackexchange.com/docs

Домашнее задание

1. Доработать код с лекции. На ЯндексДиске создать директорию <Текущая дата>, перед загрузкой файла проверять наличие его на диске. Отправить файлы на почту.

2. Есть API с информацией о супергероях (https://superheroapi.com/). Нужно определить кто самый умный(intelligence) из трех супергероев- Hulk, Captain America, Thanos. Для определения id нужно использовать метод */search/name*